Excelerate Connect - Week 3 Deliverable

Mobile App Development with Flutter Virtual Internship

Project information	Details
Student	Francis Kwarteng
Internship	Mobile App Development with Flutter Virtual Internship
Deliverable	Week 3 - Dynamic Data and Functional Forms
Application	Excelerate Connect

GitHub repository
https://github.com/franciskwarteng677/excelerate-connect

Local functional prototype - bundled JSON data and simulated form submissions only.

Week 3 Objective

The Week 3 objective was to move the Week 2 static user-interface prototype into a more functional and interactive Flutter application while keeping all data and submissions local. This stage introduced bundled JSON program data, asynchronous loading states, dynamically populated Program Listing and Program Details screens, and validated registration and feedback form simulations.

The implementation remains a learning prototype. It does not connect to an API, backend, database, authentication provider, registration service, feedback service, or persistent storage.

Summary of Changes from Week 2

Week 2 established the responsive Login, Home, Program Listing, and Program Details screens with local Dart sample records and informational prototype actions. Week 3 builds on that foundation by:

- Moving the runtime program catalogue into a bundled JSON asset.
- Adding robust JSON-to-Program parsing.
- Introducing a reusable repository contract and asset-backed implementation.
- Displaying loading, successful, source-empty, filtered-empty, and error states.
- Providing Retry when local program loading fails.
- Populating Home featured programs, Program Listing, and Program Details from the same repository-backed source.
- Replacing registration and feedback messages with functional local forms.
- Adding inline validation, submission progress, duplicate-submission prevention, success feedback, and complete form clearing.
- Expanding automated coverage for JSON parsing, the real asset, asynchronous states, navigation, filtering, and forms.

All Week 1 and Week 2 documents, wireframes, and screenshots remain historical artifacts and are preserved separately.

Dynamic Program Data Integration

Bundled JSON Catalogue

The runtime programme catalogue is stored in assets/data/programs.json. The file is declared as a Flutter asset in pubspec.yaml, allowing it to be bundled with the application and read without network access.

The catalogue contains four illustrative records:

1. Flutter Foundations
2. Career Readiness Sprint
3. Data Insights Starter
4. Project Leadership Lab

Every entry provides an identifier, title, category, short description, full description, sample deadline, duration, delivery format, eligibility requirements, and a safe local visual identifier. These records demonstrate application behaviour only and are not live or currently available programme listings.

Program Model and JSON Parsing

The reusable Program model represents the catalogue data as typed Dart objects. Program.fromJson validates every required string, rejects missing or blank values, verifies that eligibility requirements are a non-empty list of non-empty strings, and converts the JSON visual identifier into a supported ProgramVisual value.

Unsupported visual identifiers and invalid fields produce FormatException details. Successfully parsed eligibility collections and repository results are exposed as unmodifiable lists, reducing accidental changes after loading.

Program Repository and Loading Process

The program repository separates asset loading and parsing from the screens. The production AssetProgramRepository:

1. Applies an intentional delay of approximately one second so the loading state is visible.
2. Reads assets/data/programs.json through Flutter's rootBundle.
3. Decodes the JSON array.
4. Validates each entry through Program.fromJson.
5. Returns a typed List of Program objects.
6. Wraps asset and parsing failures in a clear ProgramRepositoryException.

The repository contains no network request, API key, credential, or external service integration.

Dynamic Program Listing and Details Flow

The implemented data flow is:

```
assets/data/programs.json
    -> rootBundle
    -> AssetProgramRepository
    -> Program.fromJson
    -> typed List<Program>
    -> Home Featured Programs and Program Listing
    -> selected Program
    -> Program Details
```

The Program Listing cards are built from the loaded Program objects rather than from hardcoded records inside the screen. Search continues to match title, category, short description, and full description without case sensitivity. Category filters, Reset, filtered result counts, and the no-results state operate on the loaded collection.

Opening a card or its View details action passes that selected Program object to Program Details. The details screen dynamically displays its visual, category, title, short and full descriptions, deadline, duration, delivery format, and eligibility requirements.

Program Details is pushed over the existing Program Listing route. Normal Back navigation therefore returns to the same listing instance and preserves the learner's current search text and category selection.

State Management Approach

Week 3 uses Flutter's built-in StatefulWidget and setState approach. A repository interface is injected into Home and Program Listing, which keeps the production code testable without adding a state-management dependency.

Each screen tracks its current program collection, loading flag, and loading error. A load-generation value prevents an older asynchronous request from overwriting the result of a newer request if the repository instance changes.

No Provider, Riverpod, database-backed state, or persistent application state was introduced.

Loading, Empty, and Error States

Loading Indicators

Program Listing displays a centred CircularProgressIndicator while the catalogue is loading. Home uses a compact loading indicator inside Featured Programs so the welcome banner, search entry point, events, announcement, quick links, and bottom navigation remain available.

Registration and feedback submissions also display progress indicators during their simulated delays.

Empty States

Two catalogue-empty situations are handled separately:

- A source-empty state appears if the JSON file loads successfully but contains no program records.
- A filtered-empty state appears when the loaded catalogue contains programs but none match the current search and category.

The filtered-empty card offers a Show all programs action that resets the active search and filters.

Error Handling and Retry

If the asset cannot be read or parsed, Program Listing displays a clear error card instead of program cards. Its Retry button starts a new repository load. Home provides a compact error card and Retry action for the Featured Programs section.

The interface does not disguise local loading failures as live-service errors, because no remote service is involved.

Registration Form

The Apply or register action in Program Details opens a responsive local registration form for the selected program.

Fields

- Full name
- Email address
- Education level dropdown
- Motivation or reason for joining

- Agreement checkbox confirming that the learner understands the local prototype limitation

Validation

- Full name is required.
- Email is required and must have a reasonable format.
- Education level must be selected.
- Motivation is required and must contain at least 20 characters.
- The local-prototype agreement must be selected.

Invalid values produce inline messages beside the relevant controls.

Feedback Form

The Give feedback action opens a responsive local feedback form while retaining the selected program context.

Fields

- Full name
- Email address
- Experience rating dropdown
- Comments

Validation

- Full name is required.
- Email is required and must have a reasonable format.
- An experience rating must be selected.
- Comments are required and must contain at least 20 characters.

The form presents inline validation errors and does not submit until all requirements are satisfied.

Simulated Submission, Success, and Form Clearing

Both forms use an approximately 900-millisecond local delay to demonstrate an in-progress submission state. While that delay is active:

- Form controls are disabled.
- The submit action is disabled to prevent duplicate submissions.
- A CircularProgressIndicator and submission label are displayed.

After the simulated delay, each form displays a success SnackBar and a persistent success panel. The messages explicitly state that nothing was sent to a server or stored persistently. Text fields, dropdown selections, and the registration agreement checkbox are then cleared.

Entered information exists only temporarily in the running widget state. It is discarded after the simulated success or when the application state is closed.

Testing and Validation Performed

The completed Week 3 implementation was validated with:

- Dart formatting across 23 Dart files, with no remaining formatting changes.
- flutter analyze, which completed with no issues.
- flutter test, with all 34 tests passing.
- flutter build web, which completed successfully.
- Flutter's WebAssembly dry-run check, which also succeeded.
- JSON verification confirming all four required program records and fields.
- Asset-backed testing that loads the real bundled programs.json file.

Automated coverage includes:

- Valid and invalid Program JSON parsing.
- Unknown visual identifiers and invalid eligibility data.
- Typed immutable repository results.
- Real bundled asset loading.
- Loading, success, source-empty, error, and Retry behaviour.
- Search, category filtering, Reset, and filtered-empty behaviour after loading.
- Selected-program navigation and listing-state preservation.
- Registration and feedback validation.
- Visible disabled submission states and progress indicators.
- Successful local submissions and complete form clearing.

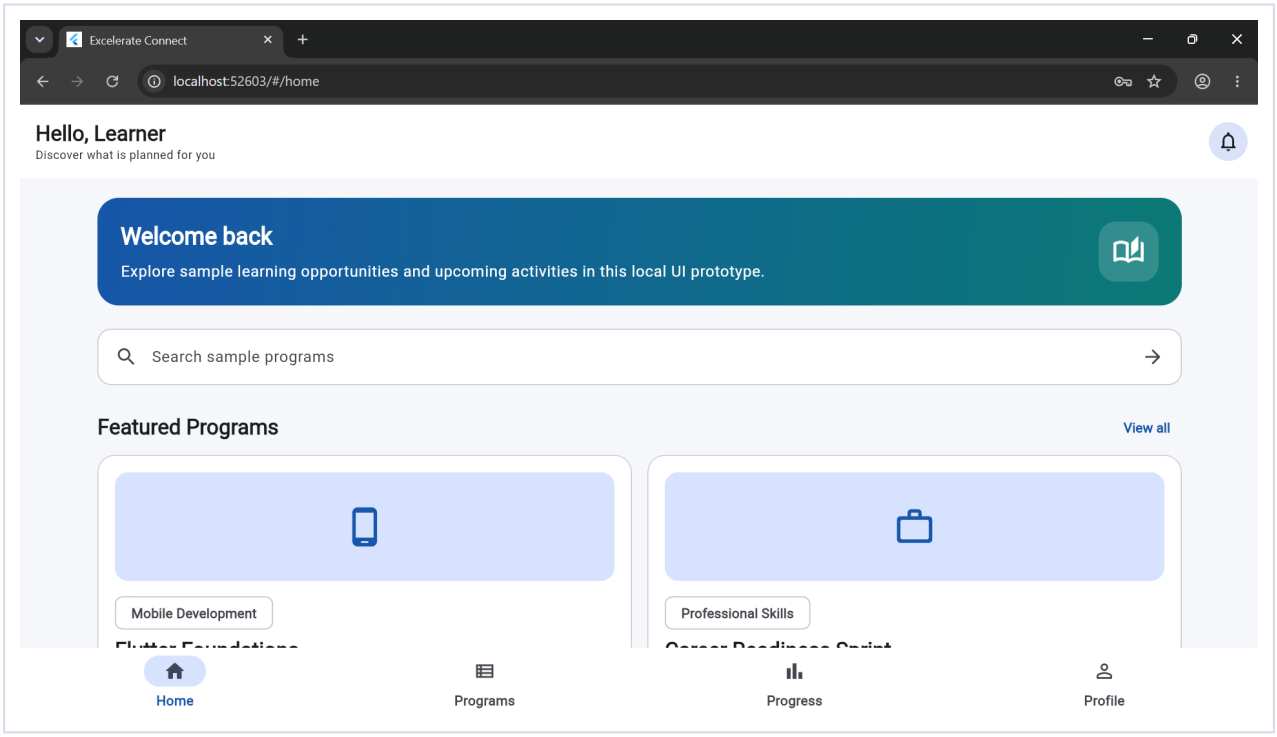
Current Limitations

- Login remains a client-side validation prototype; there is no real authentication, account creation, password recovery, or session service.
- Program data is bundled JSON, not a live catalogue or API response.
- Registration does not enroll a learner or send an application.
- Feedback is not transmitted, persisted, or available to an administrator.
- No API, backend, database, cloud service, analytics service, or persistent local storage is connected.
- Events and announcements remain static sample content.
- Progress, Profile, Notifications, account management, and administrator workflows remain planned.
- The one-second loading delay and 900-millisecond submission delays are intentional simulations.
- No official logo or unprovided Excelerate brand asset is used.

Week 3 Screenshot Gallery

The following screenshots document the working local Week 3 prototype in numerical order. They show illustrative program and form data only.

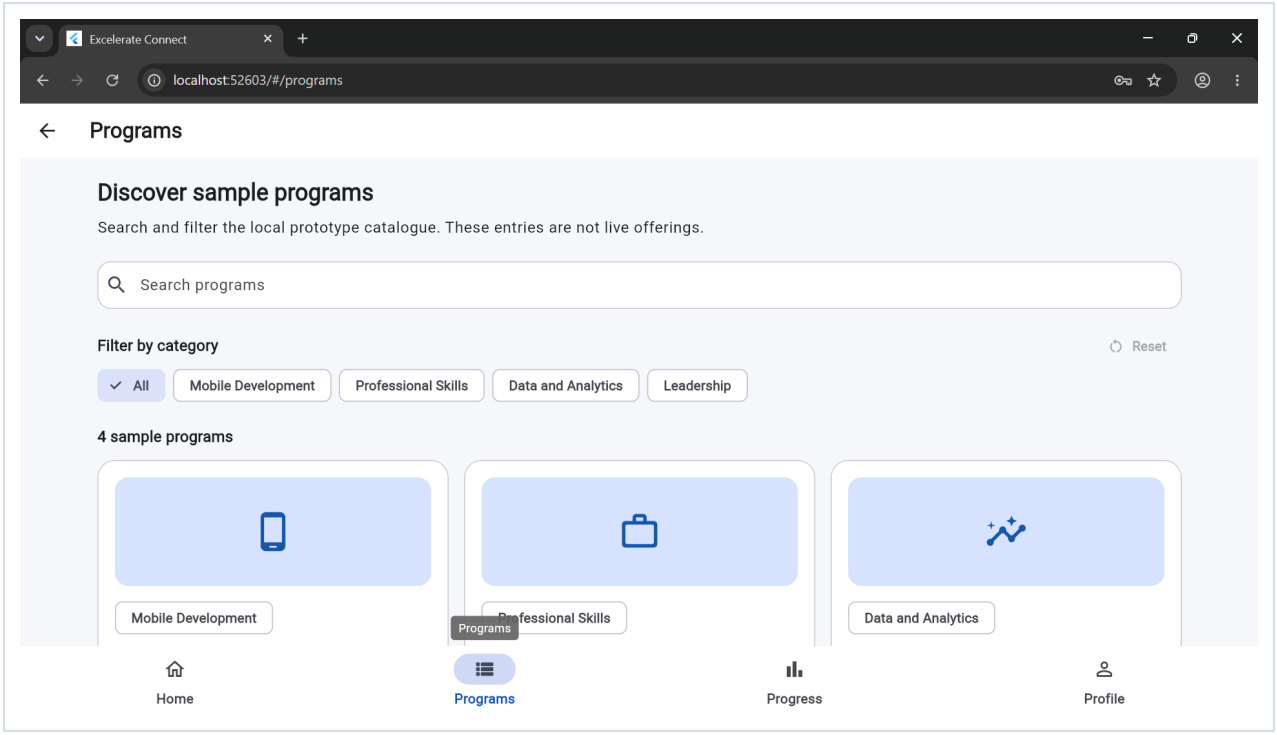
1. Home - Dynamic Programs



Caption: Home displaying featured programs loaded from the bundled JSON catalogue.

This screenshot demonstrates that the existing Home layout is preserved while Featured Programs is populated through the shared repository-backed data source.

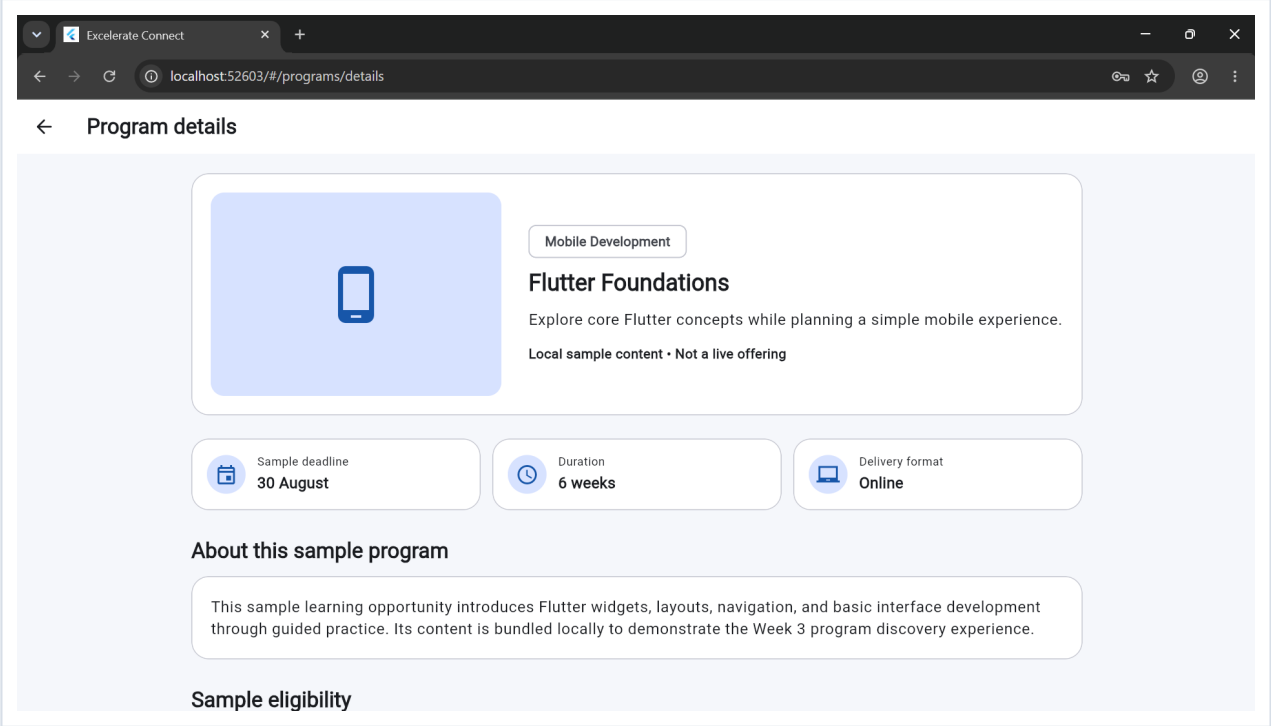
2. Program Listing - JSON Data



Caption: Searchable and filterable catalogue built from loaded Program objects.

This view demonstrates the dynamic program cards, result count, search field, category controls, and Reset action after successful local JSON loading.

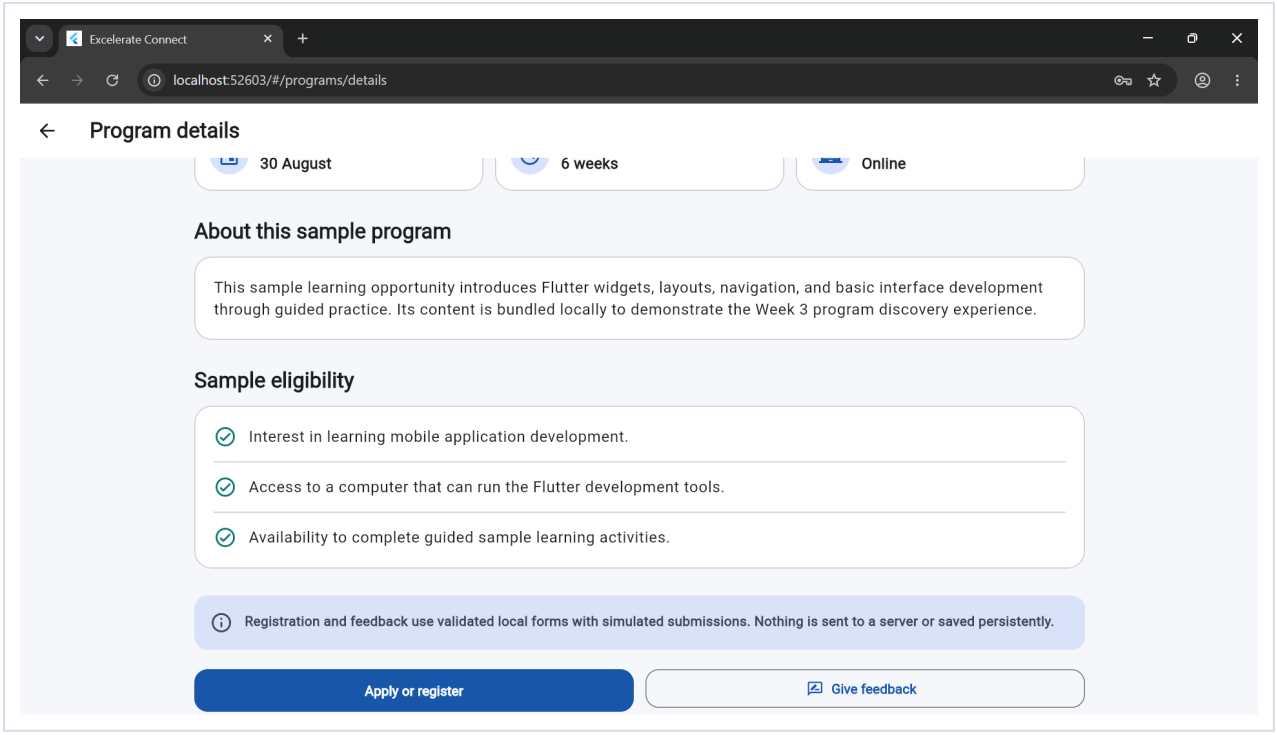
3. Program Details - Top



Caption: Selected-program identity, summary, and metadata.

The screenshot shows the selected Program object's visual, category, title, short description, deadline, duration, and delivery format.

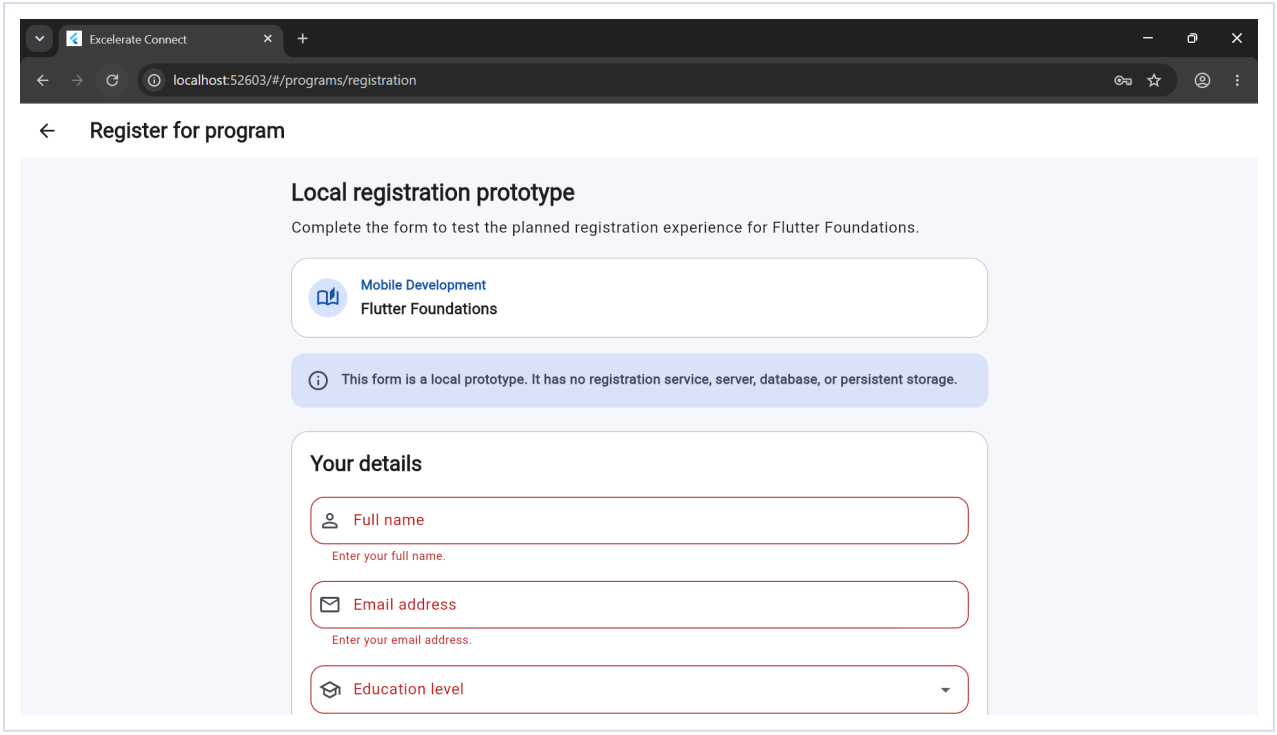
4. Program Details - Eligibility and Actions



Caption: Dynamic eligibility requirements and local form entry points.

This lower details view demonstrates the eligibility collection and the connected Apply or register and Give feedback actions.

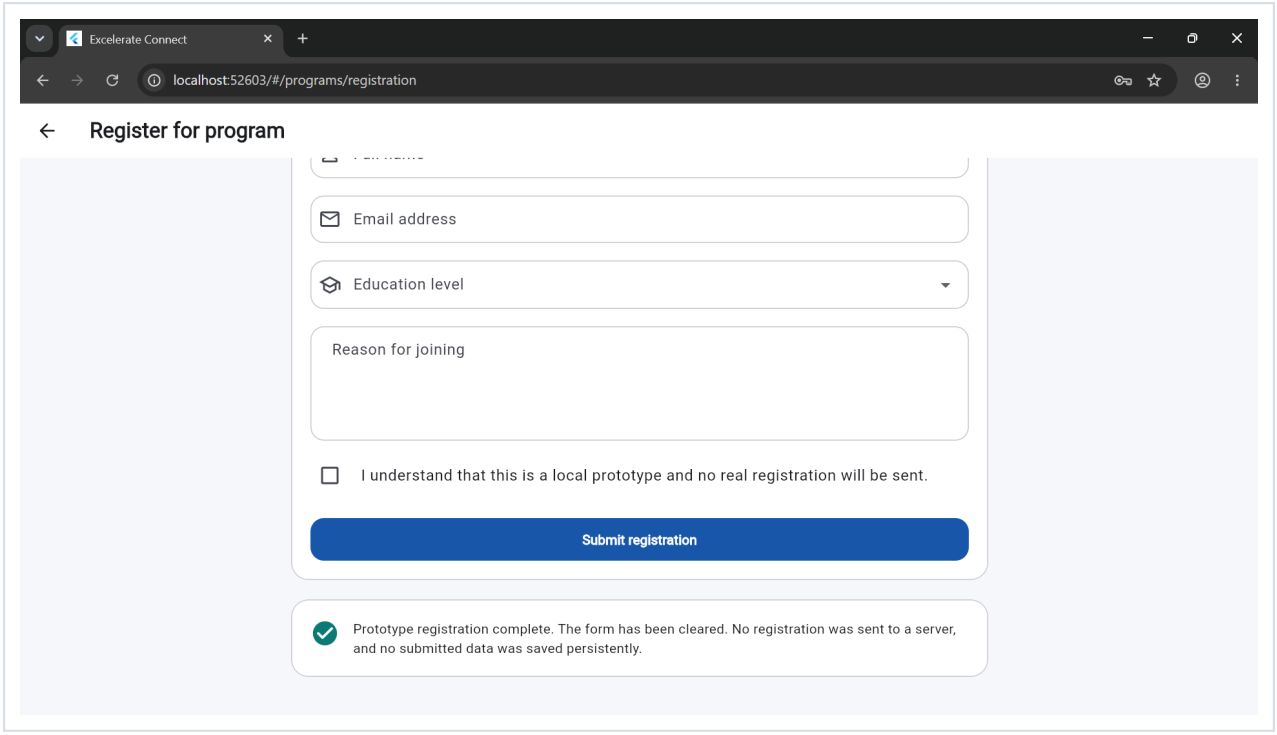
5. Registration Validation



Caption: Inline registration errors after an invalid local submission.

This screenshot demonstrates required-field, selection, motivation-length, and agreement validation without sending any information.

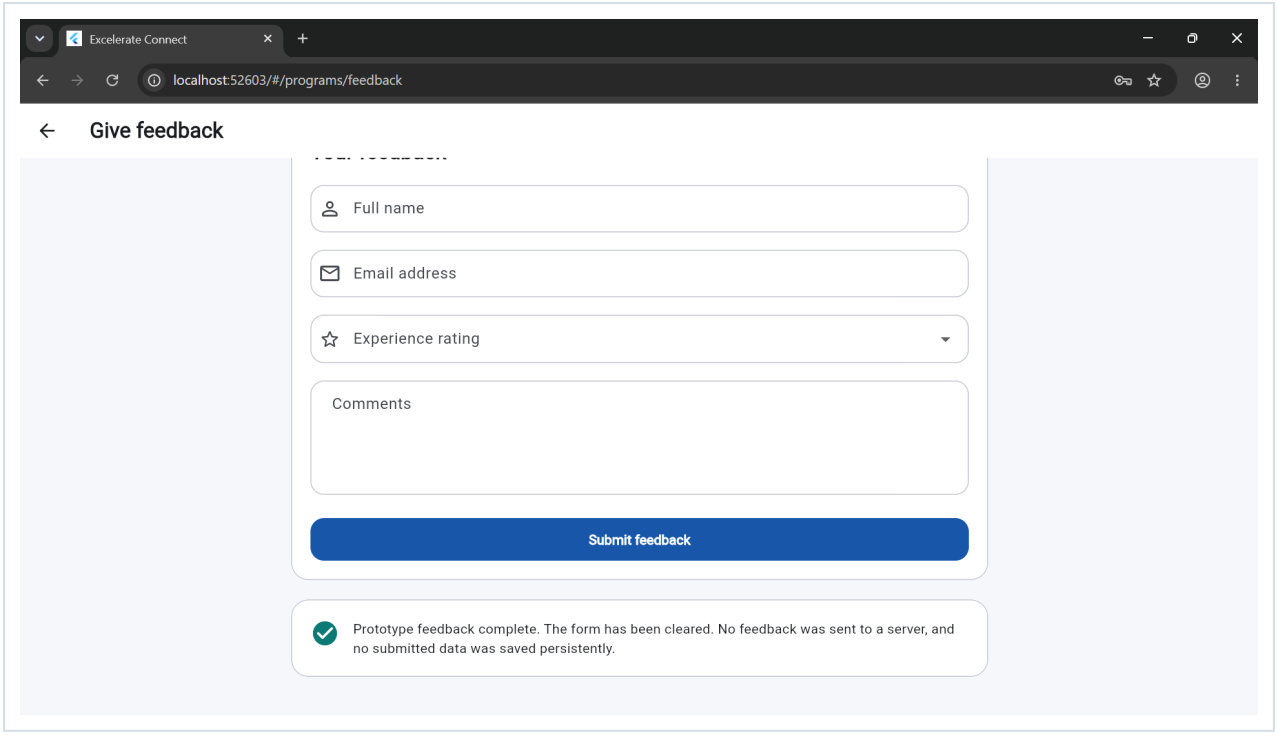
6. Registration Success



Caption: Cleared registration form with local-only success confirmation.

The view demonstrates the post-submission success state and makes clear that no registration was sent to a server or saved persistently.

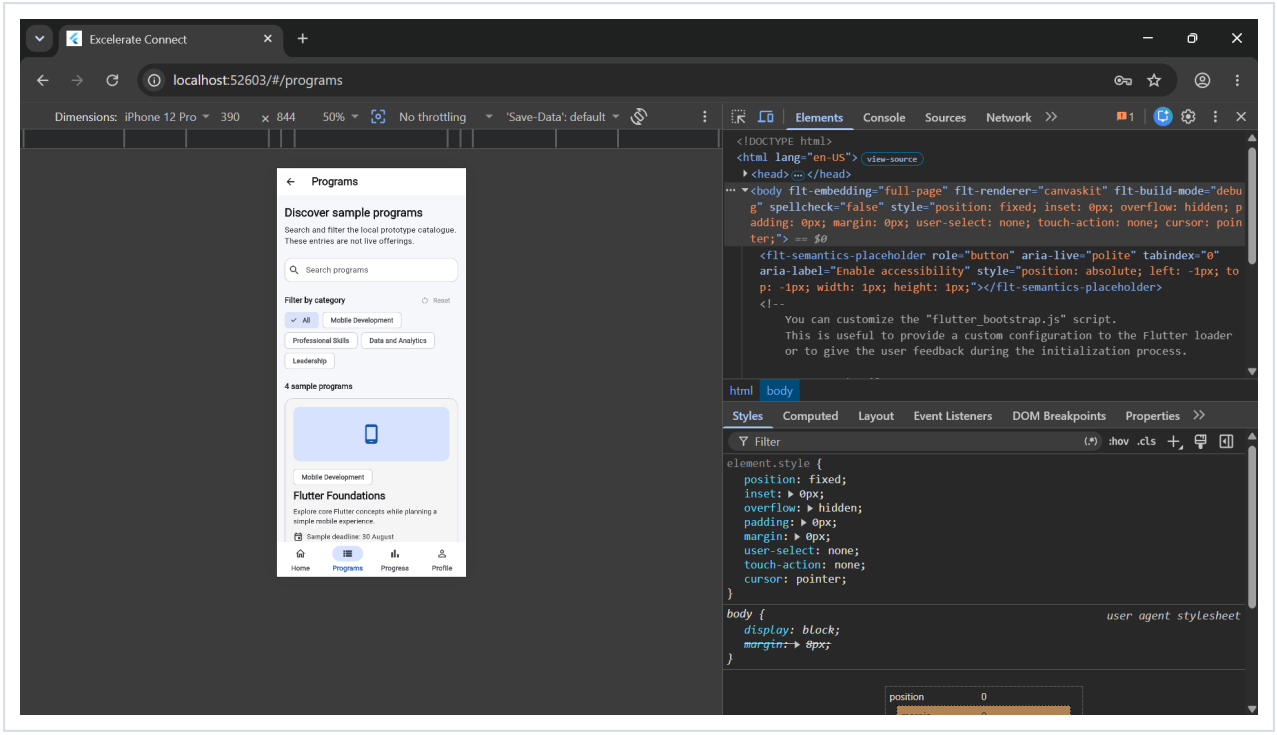
7. Feedback Success



Caption: Cleared feedback form with temporary local success confirmation.

This screenshot demonstrates successful simulated feedback handling, field clearing, and the explicit no-server limitation.

8. Mobile-Responsive Program Listing



Caption: Program Listing adapted to a narrow Chrome responsive viewport.

This view demonstrates the single-column program-card layout, wrapped filter controls, readable content, and accessible navigation at a mobile-sized width.

Conclusion

Week 3 converts the Week 2 program-discovery prototype into a repository-driven Flutter experience using bundled JSON data and explicit asynchronous UI states. Learners can search and filter loaded programs, inspect dynamic details, and complete validated registration and feedback form simulations with visible progress and honest success feedback.

The implementation provides a tested foundation for later development while clearly separating functional local interactions from unavailable authentication, live data, enrollment, feedback, backend, and persistence services.